JIRA DATA CENTER · GUIDE DE DÉVELOPPEMENT

Développer un plugin Jira Data Center

Architecture, outillage, méthodologie et bonnes pratiques

Etienne FOYANG · DPI (Data Process et Innovation) · Octobre 2026

PRÉSENTATION

Sommaire

01 **Comprendre**
Ce qu'est un plugin, architecture de Jira Data Center

02 **Outiller**
Environnement de développement et structure d'un projet

03 **Développer**
Descripteur, modules, back-end Java (Spring), front-end (AUI)

04 **Livrer**
Cycle de développement, tests, packaging et déploiement

05 **Pérenniser**
Bonnes pratiques, retour d'expérience et récapitulatif

Pourquoi développer un plugin ?

- **Adapter** : faire correspondre Jira à nos processus métier
- **Automatiser** : déclencher des actions sur les tickets et les workflows
- **Intégrer** : connecter Jira aux autres applications du SI (API REST)
- **Contrôler** : ajouter des règles de validation, de conformité et de traçabilité
- **Maîtriser** : disposer du code source, sans dépendre d'une app du Marketplace

0

redémarrage de Jira pour installer un plugin

1

seul fichier .jar à livrer

100 %

des nœuds du cluster mis à jour automatiquement

Qu'est-ce qu'un plugin Jira ?

- **Plugin (ou « app »)** : archive .jar qui étend Jira sans modifier son code source
- **Technologies** : Java côté serveur, JavaScript / CSS côté navigateur
- **Framework** : Atlassian Plugins 2 (bundle OSGi + Spring)
- **Installation** : via l'UPM, menu « Gérer les applications »
- **Cycle de vie** : activation, désactivation et mise à jour à chaud

Ce qu'un plugin peut apporter

Interface

Boutons, menus, panneaux sur les tickets, pages dédiées.

Workflows

Conditions, validateurs et post-fonctions sur les transitions.

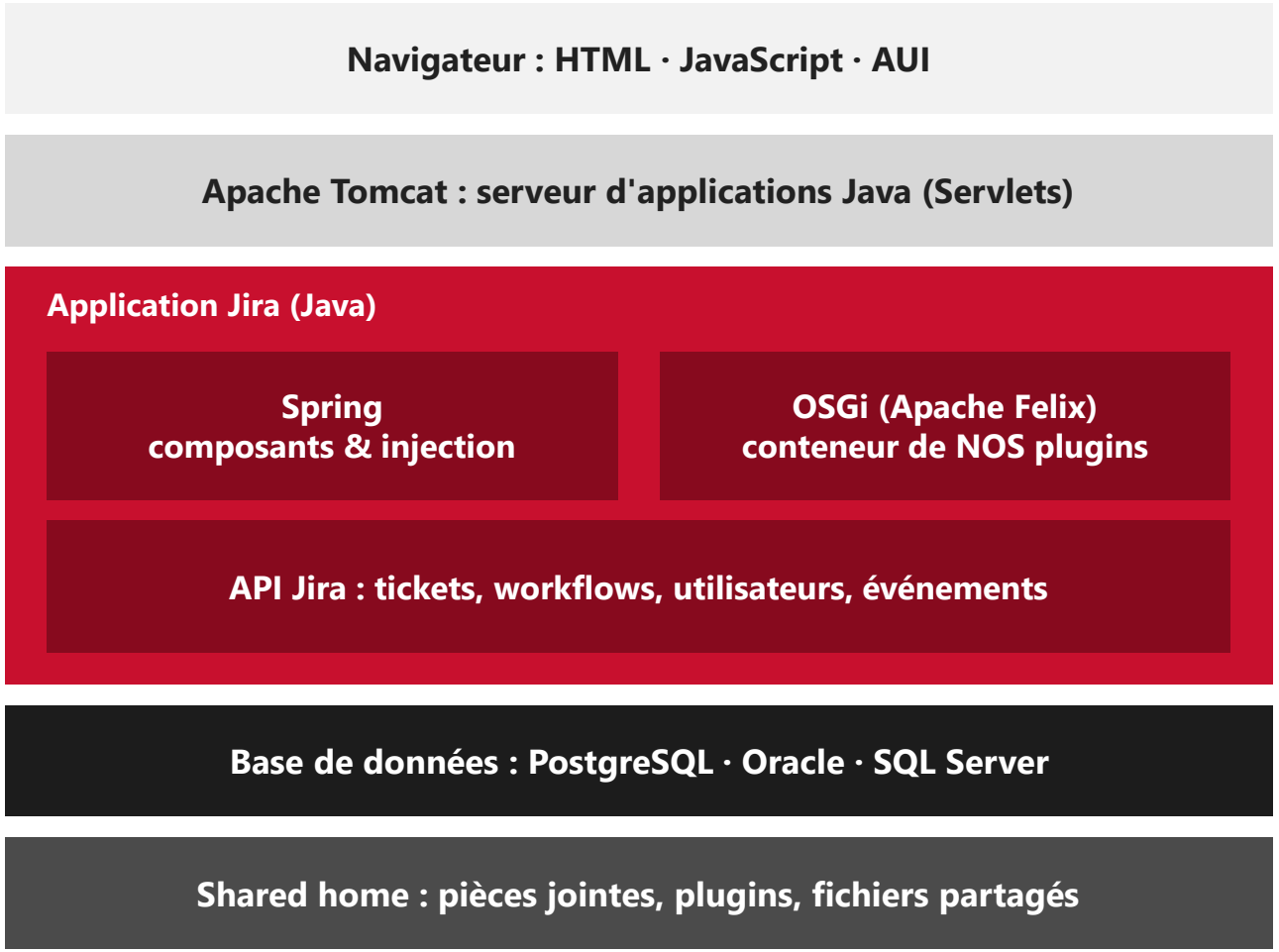
Données

Champs personnalisés, stockage dédié (Active Objects).

Intégration

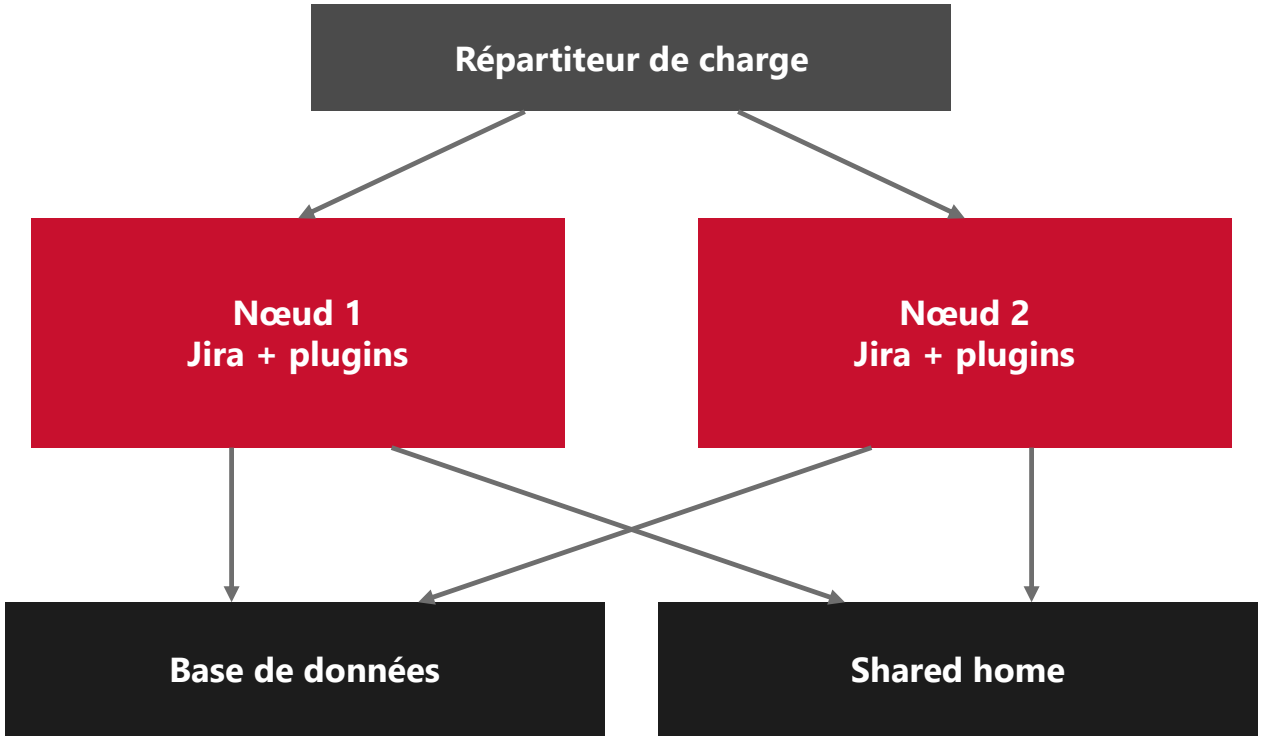
API REST, écoute d'événements, tâches planifiées, systèmes tiers.

Architecture de Jira Data Center



- **Tomcat** : reçoit les requêtes HTTP et exécute l'application
- **Spring** : instancie et relie les composants Java (injection de dépendances)
- **OSGi** : charge, isole et démarre les plugins à chaud
- **API Jira** : services métier mis à disposition des plugins
- **Base de données & shared home** : persistance commune à tous les nœuds
- Un plugin se « branche » sur l'API Jira via OSGi, sans toucher au cœur du produit

Spécificités Data Center : le cluster



Le plugin est installé une seule fois : il est automatiquement déployé sur tous les nœuds du cluster.

Ce que cela implique pour le code

- **Cluster-safe** : aucun état conservé uniquement en mémoire locale
- **Cache** : utiliser Atlassian Cache (caches répliqués)
- **Verrous** : ClusterLockService pour les traitements exclusifs
- **Tâches planifiées** : Atlassian Scheduler (exécution unique dans le cluster)
- **Fichiers** : stockés dans le shared home, jamais sur le disque local
- **Déclaration** : plugin marqué « Data Center compatible » dans son descripteur

Cycle de vie d'un plugin dans Jira



Installation à chaud

Aucun redémarrage de Jira, aucune interruption de service pour les utilisateurs.

Isolation

Chaque plugin dispose de son propre classloader : ses dépendances n'interfèrent pas avec celles des autres plugins.

Réversibilité

Désactivation ou désinstallation instantanée depuis l'administration en cas de problème.

Prérequis et outillage

JDK 17+

Version de Java supportée par la version de Jira ciblée (voir matrice de compatibilité Atlassian).

Atlassian Plugin SDK

Commandes atlas-*, Maven préconfiguré et instance Jira de test locale.

Maven

Build, gestion des dépendances et packaging via le fichier pom.xml.

IDE Java

IntelliJ IDEA (recommandé), Eclipse ou VS Code avec extensions Java.

Git

Versionnement du code, revues (pull requests), traçabilité des livraisons.

Instance de recette

Jira Data Center iso-production pour valider avant la mise en production.

Retour d'expérience

Sous Windows, installer le SDK dans un chemin sans espace (ex. C:\atlassian-sdk). Un chemin du type « C:\Program Files » provoque des erreurs Maven au lancement des commandes atlas-*.

Étape 1 : créer le projet

terminal

```
> atlas-create-jira-plugin

Define value for groupId    : com.mabanque.jira.plugins
Define value for artifactId : jira-clarification-loop
Define value for version    : 1.0.0-SNAPSHOT
Define value for package    : com.mabanque.jira.plugins.clarification

[INFO] BUILD SUCCESS
```

pom.xml

```
<properties>
    <!-- Version de Jira ciblée (celle de la production) -->
    <jira.version>10.3.5</jira.version>
</properties>
```

1 **Générer le squelette**

La commande interactive crée un projet Maven complet et fonctionnel.

2 **Identifier le plugin**

groupId + artifactId forment la clé unique du plugin (ex. jira-clarification-loop).

3 **Cibler la bonne version**

Aligner jira.version sur la version de Jira de production pour garantir la compatibilité.

4 **Vérifier**

atlas-run démarre un Jira local avec le plugin déjà installé.

Structure d'un projet de plugin

arborescence

```
jira-clarification-loop/
├── pom.xml
└── src/
    ├── main/
    │   ├── java/com/mabanque/.../clarification/
    │   │   ├── api/        ClarificationService, ...
    │   │   ├── impl/       ClarificationServiceImpl
    │   │   └── rest/       ClarificationRestResource
    │   └── resources/
    │       ├── atlassian-plugin.xml
    │       ├── js/         clarification.js
    │       ├── css/        clarification.css
    │       └── templates/  clarification-panel.vm
    └── test/
        ├── java/ut/        Tests unitaires
        └── java/it/        Tests d'intégration
```

pom.xml

Configuration Maven : version de Jira ciblée, dépendances, instructions OSGi.

atlassian-plugin.xml

Descripteur : déclare à Jira tous les modules fournis par le plugin (panneau, REST, JS/CSS).

Code & ressources

Classes Java (service métier, API REST), JavaScript / CSS et template Velocity (.vm).

Le descripteur atlassian-plugin.xml

atlassian-plugin.xml

```xml
<atlassian-plugin key="com.mabanque.jira.plugins.jira-clarification-loop"
                  name="Demandes d'informations supplémentaires" plugins-version="2">
  <plugin-info>
    <version>${project.version}</version>
    <vendor name="DPI - Data Process et Innovation" url="https://www.mabanque.fr"/>
    <param name="atlassian-data-center-status">compatible</param>
    <param name="atlassian-data-center-compatible">true</param>
  </plugin-info>

  <!-- Ressources JS / CSS chargées sur la vue d'un ticket -->
  <web-resource key="clarification-resources-v4">
    <resource type="download" name="clarification.js"  location="/js/clarification.js"/>
    <resource type="download" name="clarification.css" location="/css/clarification.css"/>
    <context>jira.view.issue</context>
  </web-resource>

  <!-- Panneau latéral droit sur les tickets + API REST -->
  <web-panel key="panneau-demande-infos-sup" location="atl.jira.view.issue.right.context">
    <resource name="view" type="velocity" location="templates/clarification-panel.vm"/>
  </web-panel>
  <rest key="clarification-rest" path="/clarification" version="1.0"/>
</atlassian-plugin>
```

- **Rôle** : « carte d'identité » lue par Jira à l'installation
- **plugin-info** : version, éditeur (DPI), compatibilité Data Center
- **web-resource** : charge clarification.js et clarification.css sur la vue ticket (jira.view.issue)
- **web-panel (panneau-demande-infos-sup)** : affiche le panneau « Demandes d'informations supplémentaires » à droite du ticket
- **rest** : expose les endpoints /rest/clarification/1.0/...

Catalogue des principaux modules

Catégorie	Module(s)	Exemple d'usage pour la banque
Interface	web-item, web-section, web-panel	Bouton dans la navigation, panneau sur un ticket
Interface	web-resource	Charger JS/CSS, composants AUI
Workflow	workflow-condition / validator / function	Bloquer une transition sans validation conformité
Données	customfield-type	Champ « Niveau de risque »
Données	ao (Active Objects)	Stocker un historique d'audit propre au plugin
Intégration	rest	API /rest/mabanque/1.0/... pour un outil tiers
Intégration	Listener d'événements (Spring)	Notifier ou tracer à la clôture d'un ticket
Recherche	jql-function	Fonction JQL métier (ex. tickets audités)
Administration	servlet, webwork	Page de configuration du plugin
Traitements	job (Atlassian Scheduler)	Synchronisation ou purge nocturne

Back-end Java : Spring et injection de dépendances

ClarificationServiceImpl.java

```java
@ExportAsService({ClarificationService.class})
@Named("clarificationService")
public class ClarificationServiceImpl implements ClarificationService {

    private final PluginSettingsFactory pluginSettingsFactory;
    private final IssueManager issueManager;
    private final CommentManager commentManager;
    private final PermissionManager permissionManager;

    @Inject
    public ClarificationServiceImpl(
            @ComponentImport PluginSettingsFactory psf,
            @ComponentImport IssueManager im,
            @ComponentImport CommentManager cm,
            @ComponentImport PermissionManager pm) {
        this.pluginSettingsFactory = psf;
        this.issueManager = im;
        this.commentManager = cm;
        this.permissionManager = pm;
    }
    // Logique métier : contrôle des droits, réassignation, historique...
}
```

- **Spring Scanner** : détecte les annotations à la compilation
- **@Named** : déclare un composant géré par Spring
- **@ExportAsService** : expose le service métier aux autres composants
- **@ComponentImport** : récupère un service fourni par Jira (via OSGi)
- **@Inject** : Spring fournit les dépendances au constructeur
- **Services Jira injectés** : IssueManager, CommentManager, PermissionManager, PluginSettingsFactory

Exposer une API REST (standards Java EE)

ClarificationRestResource.java

```java
@Path("/issue")
@Consumes({MediaType.APPLICATION_JSON})
@Produces({MediaType.APPLICATION_JSON})
public class ClarificationRestResource {

    private final ClarificationService clarificationService;
    private final JiraAuthenticationContext authContext;

    @Inject
    public ClarificationRestResource(ClarificationService cs,
            @ComponentImport JiraAuthenticationContext auth) {
        this.clarificationService = cs;
        this.authContext = auth;
    }

    @POST @Path("/{issueKey}/request")
    public Response requestClarification(
            @PathParam("issueKey") String key, RequestPayload body) {
        ApplicationUser caller = authContext.getLoggedInUser();
        return Response.ok(clarificationService.requestClarification(
            key, caller, body.recipientUsername, body.question)).build();
    }
}
```

atlassian-plugin.xml

```xml
<rest key="clarification-rest" path="/clarification" version="1.0">
    <description>API REST pour les demandes d'informations supplémentaires</description>
</rest>
```

URL exposée
/rest/clarification/1.0/issue/{key}

- **Standards** : JAX-RS (REST), Servlets, injection JSR-330
- **Format** : échanges JSON (état du ticket, permissions, historique)
- **Authentification** : utilisateur courant via JiraAuthenticationContext
- **Autorisations** : contrôle strict côté serveur (assigné, destinataire, auteur)
- **Validation** : erreurs structurées par champ renvoyées au formulaire

Front-end : web-resources, AUI et templates

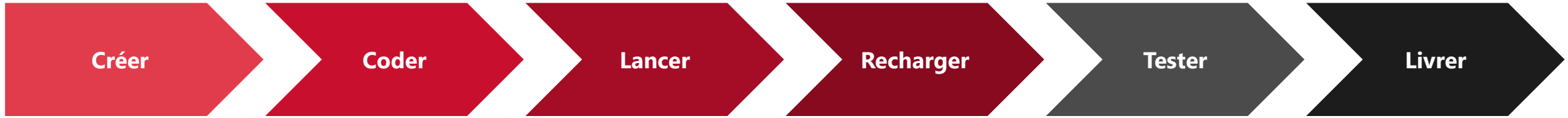
- **AUI** : bibliothèque d'interface officielle Atlassian, pour un rendu natif Jira (modales Dialog2, badges, formulaires)
- **Web-resources** : regroupent JS/CSS et les chargent selon un contexte
  - jira.view.issue (vue d'un ticket), atl.general, atl.admin...
- **Templates serveur** : Velocity (clarification-panel.vm) pour le HTML du panneau et des modales
- **Recherche dynamique** : autocomplétion d'utilisateurs via /rest/api/2/user/picker
- **Pièces jointes** : envoi multipart vers l'API native /attachments de Jira

[Insérer capture d'écran]
Panneau latéral & modale AUI « Demande d'informations supplémentaires »

clarification.js

```
$(document).on('click', '#clr-btn-open-request', function (e) {
  e.preventDefault();
  clearFormError('request');
  AJS.dialog2('#clr-request-dialog').show();
});
// Autocomplétion globale via /rest/api/2/user/picker
```

Le cycle de développement



`atlas-create-jira-plugin`	`IDE + Git`	`atlas-run`	`QuickReload`	`atlas-unit-test`	`atlas-package`
Squelette du projet	Java, JS, XML	Jira local :2990	Rechargement à chaud	Unitaires & intégration	.jar vers UPM

Boucle de développement rapide

atlas-run démarre Jira une seule fois. Ensuite, QuickReload réinstalle automatiquement le plugin dès que le .jar est recompilé (ex. atlas-package -DskipTests ou mvn package -DskipTests), sans redémarrer Jira.

Débogage

atlas-debug lance Jira avec un débogueur Java distant (port 5005) : points d'arrêt et inspection des variables directement depuis l'IDE.

Premier lancement : téléchargement de Jira et des dépendances (plusieurs minutes, environ 500 Mo dans target/, à exclure de Git).

Commandes essentielles du SDK Atlassian

Commande	Rôle
`atlas-create-jira-plugin`	Génère le squelette complet du projet
`atlas-create-jira-plugin-module`	Ajoute un module (REST, listener, workflow, champ...)
`atlas-run`	Lance un Jira local avec le plugin installé (http://localhost:2990/jira)
`atlas-debug`	Idem, avec débogueur Java distant sur le port 5005
`atlas-package`	Compile, teste et produit le livrable .jar dans target/
`atlas-unit-test`	Exécute les tests unitaires
`atlas-integration-test`	Exécute les tests d'intégration dans un Jira réel
`atlas-mvn`	Maven avec la configuration Atlassian

Tests et qualité

Tests unitaires

- JUnit + Mockito
- Rapides, sans démarrer Jira
- Valident la logique métier isolée
- atlas-unit-test

Tests d'intégration

- « Wired tests » exécutés dans un Jira réel
- Valident l'intégration avec l'API Jira
- atlas-integration-test

Recette fonctionnelle

- Scénarios métier sur l'instance de recette
- Validation multi-nœuds (Data Center)
- Non-régression avant chaque livraison

Qualité du code

- Revue de code systématique via pull requests
- Analyse statique (ex. SonarQube) et scan des dépendances vulnérables
- Build et tests automatiques à chaque modification (intégration continue)

Déploiement : du poste de développement à la production

Développement
poste local (atlas-run)

→

Recette
Jira DC iso-production

→

Pré-production
validation finale

→

Production
mise à disposition

Procédure d'installation

1. Connexion avec un compte administrateur Jira
2. Administration > Gérer les applications
3. « Importer une application », puis sélection du .jar
4. Vérifier que tous les modules sont activés

Bonnes pratiques de livraison

- Artefacts versionnés et archivés (Nexus / Artifactory)
- Note de version (changelog) à chaque livraison
- Mise en production en période de faible activité

Zéro interruption

Installation à chaud, répliquée automatiquement sur tous les nœuds du cluster.

Retour arrière

Désactiver le plugin ou réinstaller la version précédente en quelques secondes.

Point de vigilance

L'import manuel de .jar peut être désactivé par défaut sur les versions récentes (paramètre système de l'UPM) : à valider avec l'exploitation.

Bonnes pratiques

Code & architecture

- Un plugin = un périmètre fonctionnel clair
- Clés et packages préfixés (com.mabanque...)
- Uniquement les API publiques de Jira
- Code cluster-safe (cache, verrous, scheduler)
- Logs via SLF4J, jamais System.out
- **À chaque montée de version de Jira** : recompiler et retester

Industrialisation

- Git + revue de code (pull requests)
- **Intégration continue** : build et tests automatiques
- Versioning sémantique (1.2.0) + changelog
- Dépôt d'artefacts (Nexus / Artifactory)
- Documentation technique et utilisateur

Sécurité & performance

- Vérifier les permissions côté serveur
- Aucun secret en dur dans le code
- Traitements lourds en asynchrone, hors des listeners
- Dépendances à jour et sans vulnérabilité connue
- Tests de charge sur la recette Data Center

Cas concret : besoin métier (« jira-clarification-loop »)

1. Le besoin fonctionnel

Lors de l'instruction d'un ticket, la personne en charge a régulièrement besoin de faire une demande d'informations supplémentaires (ou de document) à un autre collaborateur, de lui passer temporairement la main, puis de récupérer automatiquement le ticket dès que la réponse est fournie.

2. Les limites de l'approche par scripts (SIL)

La solution historique en scripts SIL imposait de créer 4 champs personnalisés techniques (previousAssignee, question...), des écrans de transition dédiés et des post-fonctions à dupliquer sur chaque workflow de chaque projet.

3. Pourquoi un plugin Java natif ?

- **Zéro modification de workflow** : un panneau latéral unique disponible immédiatement sur tous les projets
- **Aucun champ personnalisé technique** : stockage propre au plugin via SAL PluginSettings
- **Boucle d'assignation automatique** : A sollicite B (assigné -> B), B répond (assigné -> A)
- **Recherche globale** : sélection intuitive de n'importe quel utilisateur Jira avec autocomplétion
- **Pièces jointes intégrées** : dépôt direct d'un fichier lors de la demande ou de la réponse
- **Contrôle strict des droits** : seuls les acteurs légitimes peuvent agir sur le ticket

Cas concret : parcours fonctionnel et règles métier

1. Demande d'infos sup	2. Assignation	3. Réponse	4. Retour auto
L'assigné choisit un destinataire, saisit son commentaire et joint un fichier	Statut EN ATTENTE, commentaire tracé et ticket assigné au destinataire	Le destinataire saisit sa réponse et joint le justificatif attendu	Statut RÉPONDU, réponse tracée et ticket réassigné à l'auteur initial

Action	Qui est autorisé ?	Contrôles appliqués (serveur + formulaire)
Demande d'infos sup	Uniquement l'assigné actuel du ticket	Ticket ouvert, aucune demande en attente, destinataire actif avec droit BROWSE_PROJECTS et distinct de soi-même
Répondre à la demande	Uniquement le destinataire désigné	Demande en statut PENDING, commentaire obligatoire, pièce jointe optionnelle, réassignation auto à l'auteur
Annuler la demande	Uniquement l'auteur de la demande	Demande en statut PENDING, passage en CANCELLED et réassignation immédiate à l'auteur initial
Consulter le panneau	Tout lecteur du ticket	Historique complet en lecture seule + bandeau explicatif indiquant qui peut effectuer la prochaine action

Cas concret : construction pas à pas (1/2 : Back-end)

Étape 1 : Configuration Maven & OSGi (pom.xml)

Ciblage de Jira Data Center 10.3.5 et Java 17. Ajout d'Atlassian Spring Scanner 2 pour la détection des annotations (@Named, @Inject, @ComponentImport) et des dépendances spring-beans / spring-context en scope provided. Embarquement inline de Gson (<Embed-Dependency>gson;inline=true</Embed-Dependency>) pour respecter l'isolation OSGi DMZ de Jira 10.

Étape 2 : Déclaration des modules dans atlassian-plugin.xml

Déclaration de 3 modules complémentaires : (1) web-panel « panneau-demande-infos-sup » injecté dans atl.jira.view.issue.right.context pour afficher le bloc à droite du ticket, (2) web-resource « clarification-resources-v4 » chargé sur le contexte jira.view.issue, et (3) rest « clarification-rest » exposé sur /clarification en version 1.0.

Étape 3 : Couche métier Java (api/ et impl/)

Création du contrat ClarificationService, du modèle ClarificationEntry, de l'objet ClarificationPermissions et de l'exception ClarificationException (portant le champ en erreur et le code HTTP). Implémentation dans ClarificationServiceImpl : persistance JSON par ticket via PluginSettingsFactory, contrôle des droits via PermissionManager, réassignation via IssueManager et commentaire structuré via CommentManager.

Cas concret : construction pas à pas (2/2 : REST & UI)

Étape 4 : API REST (ClarificationRestResource)

- **GET /rest/clarification/1.0/issue/{key}** : retourne l'historique complet et les permissions calculées pour l'utilisateur connecté
- **POST .../request** : vérifie les droits et les champs, supporte validateOnly=true avant upload de pièce jointe
- **POST .../reply** : enregistre la réponse du destinataire et réassigne le ticket au demandeur
- **POST .../cancel** : permet à l'auteur d'annuler sa demande et de récupérer le ticket
- **Réponses d'erreur JSON normalisées** : { error, field } avec codes HTTP 400, 401 ou 403

Étape 5 : Front-end (Velocity, JS & CSS)

- **Template clarification-panel.vm** : structure du panneau « Demandes d'informations supplémentaires » et des modales AUI Dialog2
- **Recherche globale d'utilisateurs** : autocomplétion connectée à /rest/api/2/user/picker (debounce 250 ms)
- **Gestion des pièces jointes** : validation métier préalable, puis POST multipart vers /rest/api/2/issue/{key}/attachments
- **Ergonomie formulaire** : libellés épurés (Destinataire, Commentaire, Pièce jointe) et erreurs affichées sous chaque champ
- **Support de la navigation dynamique Jira** : écoute de JIRA.Events.NEW_CONTENT_ADDED

Cas concret : importer et activer le plugin dans Jira

1. **Générer le package .jar**

   Exécuter atlas-package dans le dossier du projet pour produire target/jira-clarification-loop-1.0.0-SNAPSHOT.jar.

2. **Ouvrir la gestion des applications**

   Se connecter en administrateur Jira, cliquer sur l'icône Engrenage (Administration) > Gérer les applications (Manage apps).

3. **Importer le fichier .jar**

   Cliquer sur « Importer une application » (Upload app), sélectionner le fichier .jar depuis son poste puis valider.

4. **Vérifier l'activation**

   Le plugin apparaît activé à chaud avec ses 3 modules (panneau-demande-infos-sup, web-resource, rest).

[Insérer capture d'écran 1]
Administration > Gérer les applications
Bouton « Importer une application » + sélection du fichier jira-clarification-loop-1.0.0-SNAPSHOT.jar

[Insérer capture d'écran 2]
Confirmation d'installation dans l'UPM :
Plugin activé avec ses 3 modules (3 sur 3 activés)

Cas concret : le panneau « Demandes d'informations supplémentaires »

Intégration native sur la vue d'un ticket

- **Emplacement** : injecté dans la colonne latérale droite du ticket (atl.jira.view.issue.right.context)
- **Disponibilité immédiate** : actif sur tous les projets Jira sans toucher aux workflows ni aux écrans
- Affichage adaptatif selon l'utilisateur connecté :
  - **Si l'utilisateur est l'assigné actuel du ticket** : le bouton « Demande d'infos sup » est actif
  - **Si l'utilisateur n'est pas l'assigné** : le bouton est masqué et un bandeau indique qui peut agir
  - **Si le ticket est clôturé** : aucune nouvelle demande n'est autorisée
- **Historique intégré** : toutes les demandes passées restent consultables directement dans le panneau

[Insérer capture d'écran 3]
Vue d'un ticket Jira (colonne de droite) :
Panneau « Demandes d'informations supplémentaires » vu par l'assigné
(bouton « Demande d'infos sup » disponible)

[Insérer capture d'écran 4]
Panneau « Demandes d'informations supplémentaires » vu par un tiers :
Mode lecture seule avec bandeau explicatif des droits

Cas concret : faire une demande d'infos sup (1/2)

1. **Cliquer sur « Demande d'infos sup »**

   L'assigné actuel ouvre la fenêtre modale AUI Dialog2 directement depuis le panneau du ticket.

2. **Rechercher le destinataire**

   Saisie libre du nom ou de l'identifiant dans « Destinataire » : autocomplétion globale de tous les utilisateurs Jira.

3. **Rédiger le commentaire & joindre un fichier**

   Saisie du « Commentaire » détaillant le besoin et sélection optionnelle d'un document dans « Pièce jointe ».

4. **Validation en ligne & assignation auto**

   Si un champ est invalide, l'erreur s'affiche en rouge sous le champ (sans alerte navigateur). Si tout est valide, le fichier est joint et le ticket est assigné au destinataire.

[Insérer capture d'écran 5]
Modale « Demande d'informations supplémentaires » ouverte : Recherche globale d'utilisateur (autocomplétion), champ « Commentaire » et champ « Pièce jointe »

[Insérer capture d'écran 6]
Affichage d'une erreur de validation directement sur le formulaire (ex. destinataire ou commentaire manquant)

Cas concret : répondre et retour automatique (2/2)

1. **Ticket en statut « EN ATTENTE »**

   Le ticket est désormais assigné au destinataire. Le panneau affiche la question, l'auteur, la date et le lien vers la pièce jointe.

2. **Actions réservées aux acteurs légitimes**

   Seul le destinataire voit le bouton « Répondre ». De son côté, seul l'auteur initial dispose du bouton « Annuler ».

3. **Saisir la réponse et joindre le justificatif**

   Le destinataire clique sur « Répondre », saisit son « Commentaire » de réponse et ajoute sa « Pièce jointe ».

4. **Réassignation automatique à l'auteur**

   Dès l'envoi, le statut passe à « RÉPONDU », un commentaire est tracé sur le ticket et le ticket revient automatiquement à l'auteur initial.

[Insérer capture d'écran 7]
Panneau en statut « EN ATTENTE » + modale « Répondre » :
Saisie du commentaire de réponse et ajout de la pièce jointe

[Insérer capture d'écran 8]
Résultat après réponse :
Échange en statut « RÉPONDU », commentaire horodaté et ticket automatiquement réassigné à l'auteur initial

Cas concret : difficultés rencontrées et solutions

Problème rencontré	Cause technique	Solution mise en œuvre
Erreur Maven au lancement des commandes atlas-*	Espace dans « C:\Program Files » mal géré par les scripts .bat du SDK	Utiliser un chemin sans espace ou le chemin court Windows 8.3 (C:\PROGRA~1\...)
Échec de compilation sur InitializingBean / Spring	Dépendances Spring absentes du classpath par défaut	Ajouter spring-beans et spring-context en <scope>provided</scope> dans pom.xml
Refus d'activation dans Jira 10 (This app failed to enable)	Nouvelle barrière OSGi DMZ de Jira 10 bloquant le package interne com.google.gson	Embarquer Gson en inline (<Embed-Dependency>) et exclure son import (!com.google.gson.*)
Modale AUI restant sur une ancienne version HTML	AUI Dialog2 déplace la modale dans <body> et la conserve lors des rafraîchissements AJAX	Générer les modales dans Velocity + nettoyer l'ancienne copie dans <body> à l'initialisation
Pièce jointe envoyée malgré un formulaire invalide	L'appel à l'API /attachments partait avant la validation métier du destinataire	Pré-valider via POST .../request avec validateOnly: true avant d'envoyer le fichier
Dossier target/ très volumineux (> 500 Mo)	Contient l'instance Jira locale complète téléchargée par atlas-run	Exclure systématiquement le dossier target/ dans le fichier .gitignore

En résumé : les étapes clés

1. **Besoin**
   Définir le besoin et choisir les modules adaptés
2. **Environnement**
   JDK, SDK Atlassian, IDE et Git
3. **Développement**
   Descripteur XML, code Java et front-end
4. **Tests**
   Jira local (atlas-run), tests unitaires et d'intégration
5. **Livraison**
   Package .jar, recette puis production

Pour démarrer un projet de plugin

- Un poste de développement avec JDK et SDK Atlassian
- Une instance Jira Data Center de recette
- Un dépôt Git et, idéalement, une intégration continue
- Des compétences Java, Spring et JavaScript

Merci

Questions & échanges

RESSOURCES

Documentation développeur Jira Data Center : developer.atlassian.com/server/jira/platform
Communauté développeurs Atlassian : community.developer.atlassian.com
Bibliothèque d'interface AUI : aui.atlassian.com